

PRACTICAL-4

AIM: Find the Maximum Temperature Recorded for Each Year of NCDC Data using MapReduce

PROBLEM DEFINITION:

A meteorological research organization maintains historical weather records collected from various weather stations worldwide. The organization uses Hadoop to analyze large climate datasets and study long-term temperature trends. As a Data Analytics Engineer, you are required to process the National Climatic Data Center (NCDC) dataset and identify the maximum temperature recorded for each year using MapReduce.

Task 1: Upload the NCDC weather dataset to HDFS and examine the structure of the weather records.

Task 2: Develop and implement the Mapper program to extract the year and temperature values from each record and generate intermediate key-value pairs.

Task 3: Develop and implement the Reducer program to determine the maximum temperature recorded for each year.

Task 4: Execute the MapReduce job on Hadoop and store the results in HDFS. Verify the correctness of the output generated.

Task 5: Analyze the yearly maximum temperature trends and identify the hottest year in the dataset. Discuss the advantages of using MapReduce for large-scale climate data analysis.

Supplementary Problem: Find the minimum temperature recorded for each year. Calculate average yearly temperatures. Identify the year with the highest recorded temperature overall.

CODE (SUPPLEMENTARY PROBLEMS INCLUDED):

1. mapper.py

```
#!/usr/bin/env python3 import
```

```
sys
```

```
for line in sys.stdin: line
```

```
    = line.strip() try:
```

```
        if not line:
```

```
        continue

    year      =      line[15:19]
    temperature      =
    int(line[87:92])  quality  =
    line[92]

    if temperature == 9999:

        continue

    print(f'{year}\t{temperature}')

except (ValueError, IndexError):
    continue
```

2. reducer.py

```
#!/usr/bin/env python3 import

sys

current_year = None maximum_temperature
= None

for line in sys.stdin:
    line = line.strip()

try:
    year, temperature = line.split("\t") temperature
    = int(temperature)

    if current_year == year:
        if temperature > maximum_temperature:
            maximum_temperature = temperature else:
            if current_year is not None:
                print(f'{current_year}\t{maximum_temperature}')

        current_year = year maximum_temperature
        = temperature except ValueError:
        continue

    if current_year is not None:
```

```
print(f'{current_year}\t{maximum_temperature}')
```

3. min_mapper.py

```
#!/usr/bin/env python3 import
```

```
sys
```

```
for line in sys.stdin:
```

```
    line = line.strip()
```

```
try:
```

```
    if not line:
```

```
        continue
```

```
    year = line[15:19] temperature  
    = int(line[87:92])
```

```
    if temperature == 9999:
```

```
        continue
```

```
    print(f'{year}\t{temperature}')
```

```
except (ValueError, IndexError):
```

```
    continue
```

4. min_reducer.py

```
#!/usr/bin/env python3 import
```

```
sys
```

```
current_year = None minimum_temperature  
= None
```

```
for line in sys.stdin: line
```

```
    = line.strip() try:
```

```
    year, temperature = line.split("\t") temperature  
    = int(temperature)
```

```
    if current_year == year:
```

```
        if temperature < minimum_temperature:
```

```
            minimum_temperature = temperature else:
```

```
if current_year is not None:
    print(f'{current_year}\t{minimum_temperature}')
```

```
current_year = year
minimum_temperature = temperature
```

```
except ValueError:
    continue
```

```
if current_year is not None:
    print(f'{current_year}\t{minimum_temperature}')
```

5. avg_reducer.py

```
#!/usr/bin/env python3
```

```
import sys
```

```
current_year = None
total_temperature = 0
count = 0
for line in sys.stdin:
```

```
    line = line.strip()
```

```
try:
```

```
    year, temperature = line.split("\t")
    temperature = int(temperature)
```

```
    if current_year == year:
```

```
        total_temperature += temperature
```

```
    count += 1
    else: if current_year is
```

```
not None:
```

```
        average = total_temperature / count
        print(f'{current_year}\t{average:.2f}')
```

```
    current_year = year
```

```
    total_temperature = 0
```

```
    count = 1
```

```
except ValueError:
    continue
```

```
average = total_temperature / count
print(f'{current_year}\t{average:.2f}')
```

```

kali@kali:~/Desktop$ cat -n /Desktop/1901.gz | head
1 zcat -n /Desktop/1901.gz | head
2 02902907099999101010106004+64333+023450FM-12+000599999V020781N015919999999N0000001N-00781+99999102001AD0GF1089919999999999999999
3 029029070999991010101013804+64333+023450FM-12+000599999V02081N008219999999N0000001N-00811+99999101813AD0GF1089919999999999999999
4 029029070999991010101020004+64333+023450FM-12+000599999V0209991C000019999999N0000001N-00941+99999102001AD0GF1089919999999999999999
5 0290290709999910101010206004+64333+023450FM-12+000599999V021081N008219999999N0000001N-00611+99999101813AD0GF1089919999999999999999
6 0290290709999910101010113004+64333+023450FM-12+000599999V021081N008199999999N0000001N-00561+99999101716AD0GF1089919999999999999999
7 0290290709999910101010220004+64333+023450FM-12+000599999V021081N008199999999N0000001N-00281+99999101716AD0GF1089919999999999999999
8 0290290709999910101010306004+64333+023450FM-12+000599999V0202001N008199999999N0000001N-00071+99999101701AD0GF1069919999999999999999
9 0290290709999910101010313004+64333+023450FM-12+000599999V0202301N011819999999N0000001N-00331+99999101741AD0GF1089919999999999999999
10 0290290709999910101010120004+64333+023450FM-12+000599999V0202301N011819999999N0000001N-00281+99999101741AD0GF1089919999999999999999
11 0290290709999910101010406004+64333+023450FM-12+000599999V0209991C000019999999N0000001N-00331+99999102311AD0GF1089919999999999999999
kali@kali:~/Desktop$ cat -n /Desktop/1902.gz | head
1 zcat -n /Desktop/1902.gz | head
2 03502907099999101010106004+64333+023450FM-12+000599999V0201401N011819999999N0000001N-00041+99999104055AD0GF1049919999999999999999MM1381
3 0350290709999910102010113004+64333+023450FM-12+000599999V0201401N011819999999N0000001N-01001+99999100311AD0GF1049919999999999999999MM1381
4 0350290709999910102010120004+64333+023450FM-12+000599999V0201401N011819999999N0000001N-01171+99999100121AD0GF1089919999999999999999MM1381
5 0350290709999910102010206004+64333+023450FM-12+000599999V020081N008199999999N0000001N-01611+99999100121AD0GF1089919999999999999999MM1381
6 0350290709999910102010106004+64333+023450FM-12+000599999V0208091N008199999999N0000001N-01721+99999100121AD0GF1089919999999999999999
7 02902907099999101020102020004+64333+023450FM-12+000599999V0208091N008199999999N0000001N-01781+99999100421AD0GF1089919999999999999999
8 0290290709999910102010306004+64333+023450FM-12+000599999V0209991C000019999999N0000001N-01781+99999100071AD0GF1089919999999999999999
9 035029070999991010201013804+64333+023450FM-12+000599999V0209991C000019999999N0000001N-01721+99999100901AD0GF1089919999999999999999MM1721
10 0350290709999910102010106004+64333+023450FM-12+000599999V0208161N020119999999N0000001N-01061+999999911AD0GF1089919999999999999999MM1721
kali@kali:~/Desktop$ cat -n /Desktop/1903.gz | head
1 zcat -n /Desktop/1903.gz | head
2 02902907099999101010106004+64333+023450FM-12+000599999V020781N015919999999N0000001N-00781+99999102001AD0GF1089919999999999999999
3 029029070999991010101013804+64333+023450FM-12+000599999V02081N008219999999N0000001N-00811+99999101813AD0GF1089919999999999999999
4 029029070999991010101020004+64333+023450FM-12+000599999V0209991C000019999999N0000001N-00941+99999102001AD0GF1089919999999999999999
5 0290290709999910101010206004+64333+023450FM-12+000599999V021081N008219999999N0000001N-00611+99999101813AD0GF1089919999999999999999
6 0290290709999910101010113004+64333+023450FM-12+000599999V021081N008199999999N0000001N-00561+99999101716AD0GF1089919999999999999999
7 0290290709999910101010220004+64333+023450FM-12+000599999V021081N008199999999N0000001N-00281+99999101716AD0GF1089919999999999999999
8 0290290709999910101010306004+64333+023450FM-12+000599999V0202001N008199999999N0000001N-00071+99999101701AD0GF1069919999999999999999
9 0290290709999910101010313004+64333+023450FM-12+000599999V0202301N011819999999N0000001N-00331+99999101741AD0GF1089919999999999999999
10 0290290709999910101010120004+64333+023450FM-12+000599999V0202301N011819999999N0000001N-00281+99999101741AD0GF1089919999999999999999
11 0290290709999910101010406004+64333+023450FM-12+000599999V0209991C000019999999N0000001N-00331+99999102311AD0GF1089919999999999999999
kali@kali:~/Desktop$ cat -n /Desktop/1904.gz | head
1 zcat -n /Desktop/1904.gz | head
2 03502907099999101010106004+64333+023450FM-12+000599999V0201401N011819999999N0000001N-00041+99999104055AD0GF1049919999999999999999MM1381
3 
```

```

kali@kali:~/ncdc/python$ head mapper_output.txt
1901 -76
1901 -72
1901 -94
1901 -61
1901 -56
1901 -28
1901 -47
1901 -33
1901 -28
1901 -33

kali@kali:~/ncdc/python$ cat ~/Desktop/1901.gz | ./mapper.py | sort | ./reducer.py
1901 317

kali@kali:~/ncdc/python$ hdfs dfs -put ~/Desktop/1901.gz /ncdc/input/
2026-08-14 05:22:39.560 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-08-14 05:22:39.642 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
put: /ncdc/input/1901.gz: file exists

kali@kali:~/ncdc/python$ hdfs dfs -ls /ncdc/input/
2026-08-14 05:24:16.667 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 2 items
-rw-r--r-- 1 kali supergroup 71867 2026-08-14 05:15 /ncdc/input/1901.gz
-rw-r--r-- 1 kali supergroup 71867 2026-08-14 05:15 /ncdc/input/1901.gz

kali@kali:~/ncdc/python$ find /usr/local/hadoop -name "hadoop-streaming-*.jar"
/usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.4.1.jar
/usr/local/hadoop/share/hadoop/tools/sources/hadoop-streaming-3.4.1-sources.jar
/usr/local/hadoop/share/hadoop/tools/sources/hadoop-streaming-3.4.1-test-sources.jar

kali@kali:~/ncdc/python$ hdfs dfs -rm -r /ncdc/output
2026-08-14 05:25:02.637 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
rm: /ncdc/output: No such file or directory

kali@kali:~/ncdc/python$ hdfsadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.4.1.jar \
input /ncdc/input \
output /ncdc/output \
mapper /home/kali/ncdc/python/mapper.py \
reducer /home/kali/ncdc/python/reducer.py
2026-08-14 05:25:38.071 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

kali@kali:~/ncdc/python$ hdfsadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.4.1.jar \
input /ncdc/input \
output /ncdc/output \
mapper /home/kali/ncdc/python/mapper.py \
reducer /home/kali/ncdc/python/reducer.py
2026-08-14 05:25:38.071 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
2026-08-14 05:25:38.071 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8032
2026-08-14 05:25:38.071 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8032
2026-08-14 05:25:38.071 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1786697842884_0001
2026-08-14 05:25:38.071 INFO mapreduce.JobSubmitter: Disabling feature Coding for path: /tmp/hadoop-yarn/staging/kali/.staging/job_1786697842884_0001
2026-08-14 05:25:39.147 INFO mapreduce.JobSubmitter: number of splits: 2
2026-08-14 05:25:39.212 INFO mapreduce.JobSubmitter: number of splits: 2
2026-08-14 05:25:39.451 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1786697842884_0001
2026-08-14 05:25:39.451 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1786697842884_0001
2026-08-14 05:25:39.641 INFO conf.Configuration: resource-types.xml not found
2026-08-14 05:25:39.641 INFO conf.Configuration: resource-types.xml not found
2026-08-14 05:25:40.000 INFO impl.VarClientImpl: Submitted application application_1786697842884_0001
2026-08-14 05:25:40.187 INFO mapreduce.Job: The url to track the job: http://kali:8088/proxy/application_1786697842884_0001/
2026-08-14 05:25:40.384 INFO mapreduce.Job: Running Job: job_1786697842884_0001
2026-08-14 05:25:50.988 INFO mapreduce.Job: Job job_1786697842884_0001 running in user mode : false
2026-08-14 05:25:58.988 INFO mapreduce.Job: map 0% reduce 0%
2026-08-14 05:26:02.664 INFO mapreduce.Job: map 100% reduce 0%
2026-08-14 05:26:02.926 INFO mapreduce.Job: map 100% reduce 100%
2026-08-14 05:26:08.348 INFO mapreduce.Job: Job job_1786697842884_0001 completed successfully
2026-08-14 05:26:08.483 INFO mapreduce.Job: Counters: 54

File System Counters
  FILE: Number of bytes read=140836
  FILE: Number of bytes written=214181
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=148156
  HDFS: Number of bytes written=18
  HDFS: Number of read operations=11
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=2
  HDFS: Number of bytes read erasure-coded=0

Job Counters
  Launched map tasks=2
  Launched reduce tasks=1
  Data-local map tasks=2
  Total time spent by all maps in occupied slots (ms)=18844
  Total time spent by all reduces in occupied slots (ms)=2772
  Total time spent by all map tasks (ms)=18844
  Total time spent by all reduce tasks (ms)=2772
  Total vcore-millisecs taken by all map tasks=18844
  Total vcore-millisecs taken by all reduce tasks=2772
  Total megabyte-millisecs taken by all map tasks=19296256

File System Counters
  FILE: Number of bytes read=140836
  FILE: Number of bytes written=214181
  FILE: Number of read operations=0
  FILE: Number of large read operations=0
  FILE: Number of write operations=0
  HDFS: Number of bytes read=148156
  HDFS: Number of bytes written=18
  HDFS: Number of read operations=11
  HDFS: Number of large read operations=0
  HDFS: Number of write operations=2
  HDFS: Number of bytes read erasure-coded=0

Job Counters
  Launched map tasks=2
  Launched reduce tasks=1
  Data-local map tasks=2
  Total time spent by all maps in occupied slots (ms)=18844
  Total time spent by all reduces in occupied slots (ms)=2772
  Total time spent by all map tasks (ms)=18844
  Total time spent by all reduce tasks (ms)=2772
  Total vcore-millisecs taken by all map tasks=18844
  Total vcore-millisecs taken by all reduce tasks=2772
  Total megabyte-millisecs taken by all map tasks=19296256

Map-Reduce Framework
  Map input records=13130
  Map output records=13129
  Map output bytes=13172
  Map output materialized bytes=140842
  Input split bytes=184
  Combine input records=0
  Combine output records=0
  Reduce shuffle bytes=140842
  Reduce input records=13129
  Reduce output records=2
  Spilled Records=26258
  Shuffled Maps=2
  Failed Shuffles=0
  Merged Map outputs=2
  GC time elapsed (ms)=118
  CPU time spent (ms)=1818
  Physical memory (bytes) snapshot=801280000
  Virtual memory (bytes) snapshot=8238608192
  Total committed heap usage (bytes)=43281312
  Peak Map Physical memory (bytes)=297848832
  Peak Map Virtual memory (bytes)=234561728
  Peak Reduce Physical memory (bytes)=280999552
  Peak Reduce Virtual memory (bytes)=2758373888

Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REducer=0

File Input Format Counters
  Bytes Read=147972
  File Output Format Counters
  Bytes Written=18

2026-08-14 05:26:08.455 INFO Streaming.StreamJob: Output directory: /ncdc/output

kali@kali:~/ncdc/python$ hdfs dfs -ls /ncdc/output
2026-08-14 05:26:35.680 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Found 2 items
-rw-r--r-- 1 kali supergroup 0 2026-08-14 05:26 /ncdc/output/_SUCCESS
-rw-r--r-- 1 kali supergroup 18 2026-08-14 05:26 /ncdc/output/part-00000

kali@kali:~/ncdc/python$ hdfs dfs -cat /ncdc/output/part-00000

```

```

(kali@kali)~[/ncdc/python]
$ hdfs dfs -cat /ncdc/min_output/part-00000
2026-08-14 05:35:58,826 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
1901 -333
1902 -328

(kali@kali)~[/ncdc/python]
$ nano ~/ncdc/python/avg_reducer.py

(kali@kali)~[/ncdc/python]
$ chmod +x ~/ncdc/python/avg_reducer.py

(kali@kali)~[/ncdc/python]
$ hdfs dfs -rm -r /ncdc/avg_output
2026-08-14 05:36:54,934 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
rm: '/ncdc/avg_output': No such file or directory

(kali@kali)~[/ncdc/python]
$ hadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.4.1.jar \
-file:/home/kali/ncdc/python/mapper.py,/home/kali/ncdc/python/avg_reducer.py \
-input /ncdc/input \
-output /ncdc/avg_output \
-mapper "python3 mapper.py" \
-reducer "python3 avg_reducer.py"
2026-08-14 05:37:24,945 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
packageJobJar: [/tmp/hadoop-unjar1062809550966670251/] [] /tmp/streamjob062111113655378527.jar tmpDir=null
2026-08-14 05:37:25,388 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8032
2026-08-14 05:37:25,717 INFO client.DefaultHadoopFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8032
2026-08-14 05:37:26,665 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/kali/.staging/job_1786697842804_0005
2026-08-14 05:37:28,035 INFO mapred.FileInputFormat: Total input files to process : 2
2026-08-14 05:37:28,317 INFO mapreduce.JobSubmitter: number of splits:2
2026-08-14 05:37:29,118 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1786697842804_0005
2026-08-14 05:37:29,119 INFO mapreduce.JobSubmitter: Executing with tokens: []

(kali@kali)~[/ncdc/python]
$ hdfs dfs -cat /ncdc/avg_output/part-00000
2026-08-14 05:37:58,447 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
1901 46.70
1902 21.66

(kali@kali)~[/ncdc/python]
$ hdfs dfs -cat /ncdc/output/part-00000 | sort -k2 -n | tail -1
2026-08-14 05:38:19,576 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
1901 317

(kali@kali)~[/ncdc/python]
$ hdfs dfs -cat /ncdc/min_output/part-00000
2026-08-14 05:40:18,446 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
1901 -333
1902 -328

(kali@kali)~[/ncdc/python]
$ hdfs dfs -cat /ncdc/min_output/part-00000
2026-08-14 05:40:41,880 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
1901 -333
1902 -328

(kali@kali)~[/ncdc/python]

```

KEY QUESTIONS / ANALYSIS:

1) How is weather data parsed and processed using MapReduce?

Weather data is stored in HDFS as large input files. The Mapper reads each weather record, extracts the year and temperature, and generates key-value pairs. Hadoop groups all temperature values having the same year and sends them to the Reducer. The Reducer compares the temperatures for each year and selects the maximum temperature. Finally, the yearly maximum temperatures are stored back in HDFS.

2) What key-value pairs are generated by the Mapper?

The Mapper generates the year as the key and the temperature as the value.

Format:

(year, temperature)

Example:

2020 35
2020 38
2021 36
2021 40

Here, 2020 and 2021 are keys, while the temperature values are the corresponding values.

3) How does the Reducer identify the maximum temperature for a given year?

The Reducer receives all temperature values belonging to the same year. It compares these values one by one and keeps track of the highest value.

Example:

2020 → [35, 38, 36, 40]

The Reducer selects:

2020 → 40

Thus, the output contains the maximum temperature recorded for each year.

4) What challenges arise when processing large climate datasets?

Some major challenges are:

1. Huge data volume – Climate datasets can contain billions of records.
2. Data cleaning – Missing, invalid, or inconsistent weather records may occur.
3. Processing time – Sequential processing can take a long time.
4. Data distribution – Data must be efficiently distributed across multiple nodes.
5. Fault tolerance – Hardware or node failures can occur during processing.
6. Storage management – Large datasets require significant storage capacity.

5) How does Hadoop improve scalability and performance for weather data analytics?

Hadoop improves scalability and performance by distributing large datasets across multiple machines in a cluster. HDFS provides distributed storage, while MapReduce processes data in parallel across multiple nodes. If one node fails, Hadoop can recover the data using replication and rerun failed tasks. Therefore, Hadoop can efficiently process very large climate datasets with better scalability, fault tolerance, and performance. It also reduces processing time by dividing complex tasks into smaller tasks that can run simultaneously. Hadoop can easily scale by adding more nodes to the cluster as the size of the dataset increases.

CONCLUSION:

The practical successfully demonstrated how Hadoop MapReduce can be used to process largescale NCDC weather data and find the maximum temperature recorded for each year. The Mapper extracted year-temperature key-value pairs, while the Reducer grouped the data and identified the maximum temperature for each year. This practical also showed how MapReduce can be extended to find minimum and average temperatures. Overall, Hadoop provides efficient, scalable, parallel, and fault-tolerant processing for large climate datasets.